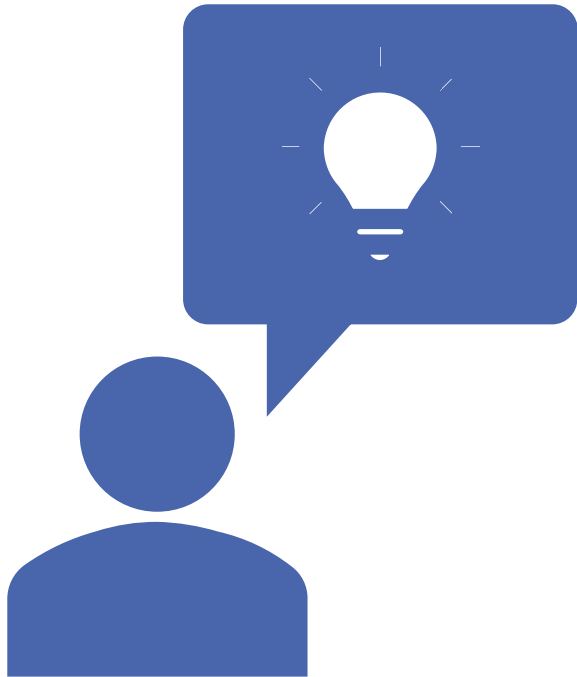


Informed Search





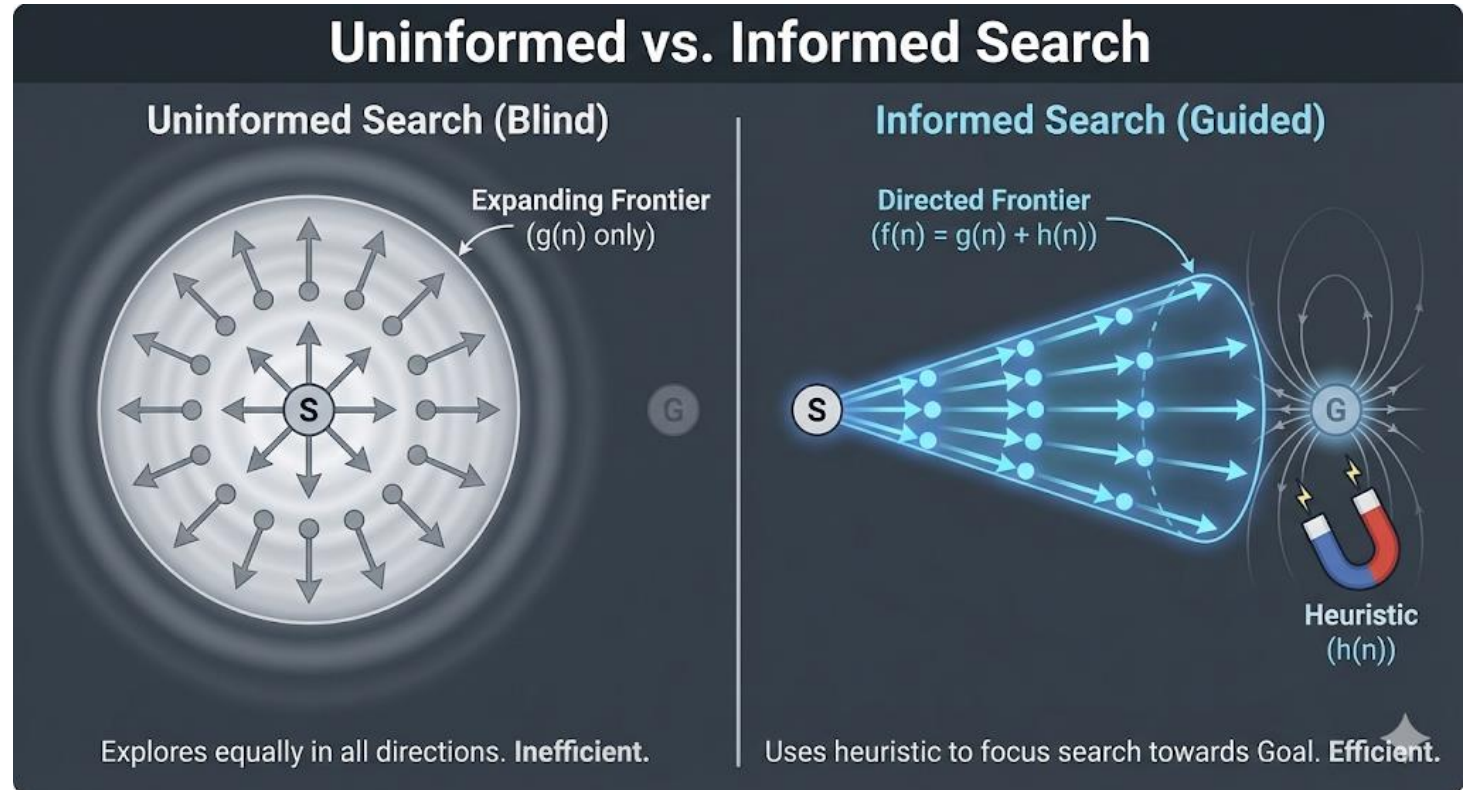
Informed Search

- Uses **problem-specific knowledge** (heuristics) to guide the search toward the goal more efficiently.
- **Key idea:**
Instead of expanding nodes in arbitrary order, **evaluate which node appears most promising** using a heuristic function $h(n)$.
- **Characteristics**
 - **Uses a heuristic** $h(n) \rightarrow$ estimates cost from node n to goal.
 - **Goal:** Reduce the number of nodes explored.
 - **More efficient** than uninformed (blind) search in many cases.
 - **Examples:** Greedy Best-First Search, A* Search.

Why it Matters

- Mimics human “educated guessing.”
- Focuses the search toward the goal.
- Performance depends heavily on heuristic quality.

Heuristics are *not guaranteed* to be correct, but a good one can drastically speed up problem solving.



A heuristic $h(n)$ is *good* if it:

- **Guides search efficiently** (fewer nodes expanded).
- **Is admissible** (never overestimates the cost to the goal).
- **Is consistent** (satisfies triangle inequality).

Goal in design:

- Minimize search cost while maintaining optimality.

Admissible heuristics

A heuristic $h(n)$ is admissible if for every node n ,

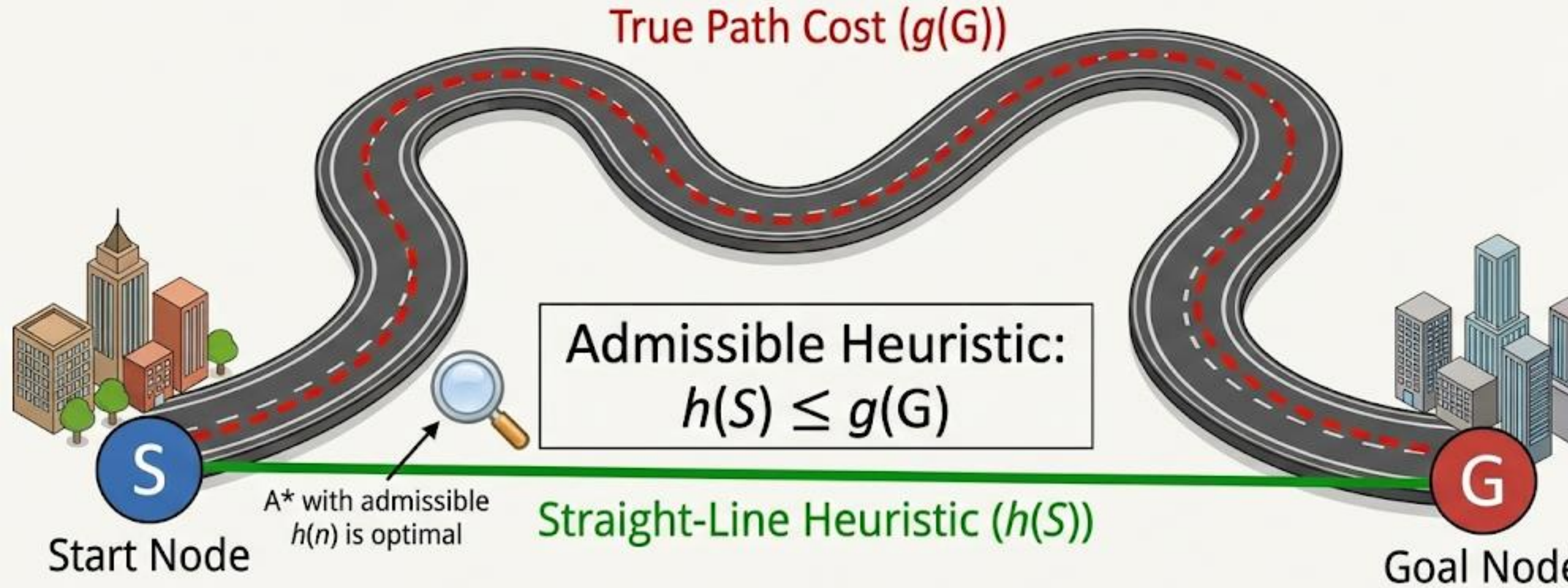
- $h(n) \leq h^*(n)$, where $h^*(n)$ is the true cost to reach the goal state from n .

An admissible heuristic never overestimates the cost to reach the goal, i.e., it is optimistic

Example: $h_{SLD}(n)$ (never overestimates the actual road distance)

Theorem: If $h(n)$ is admissible, A^* using TREE-SEARCH is optimal

Admissibility: The Straight-Line Heuristic ($h(n)$)



A heuristic is admissible if it never overestimates the true cost to the goal.
Straight-line distance is always $\leq$ true road distance.

Admissible heuristics

E.g., for the 8-puzzle:

- $h_1(n)$ = number of misplaced tiles
- $h_2(n)$ = total Manhattan distance

(i.e., no. of squares from desired location of each tile)

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

- $h_1(S) = ?$
- $h_2(S) = ?$

Consistent heuristics

- A heuristic is consistent if for every node n , every successor n' of n generated by any action a ,

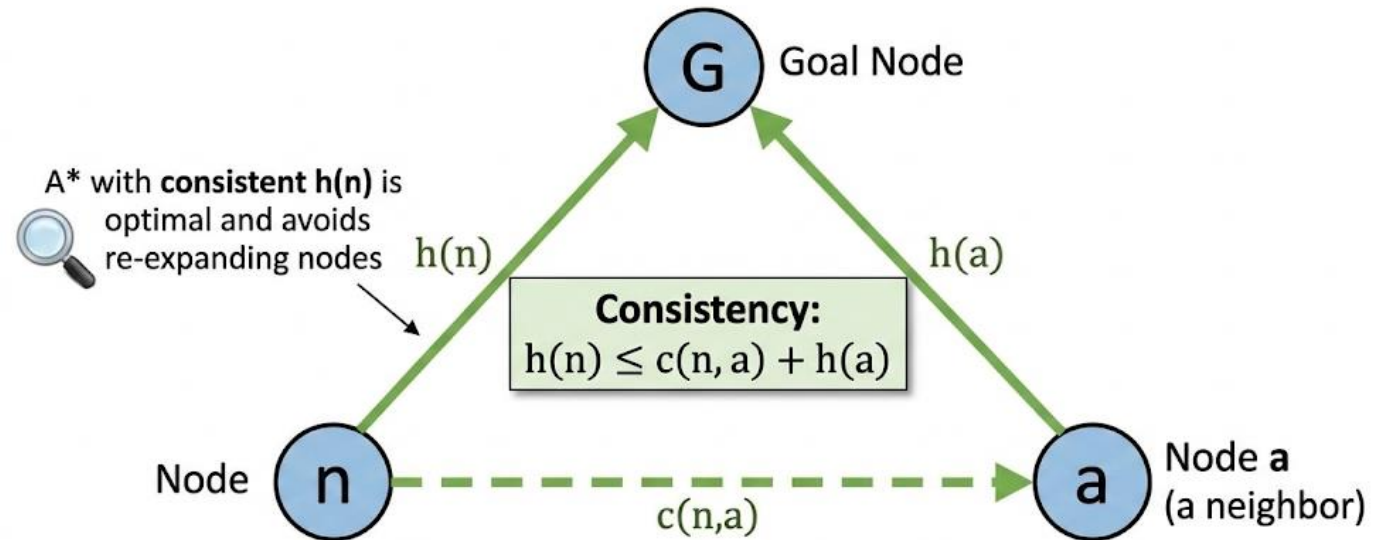
$$h(n) \leq c(n, a, n') + h(n')$$

- If h is consistent, we have

$$\begin{aligned} f(n') &= g(n') + h(n') \\ &= g(n) + c(n, a, n') + h(n') \\ &\geq g(n) + h(n) \\ &= f(n) \end{aligned}$$

- i.e., $f(n)$ is non-decreasing along any path.
- Theorem: If $h(n)$ is consistent, A* using GRAPH-SEARCH is optimal

Consistency: The Triangle Inequality of Heuristics



A heuristic is **consistent** if the estimated cost to the goal from 'n' is never more than the cost to a neighbor 'a' **plus** the estimated cost from 'a' to the goal.

Imagine you're in City A, and want to get to City C:

Direct estimate from A $\rightarrow$ C: $h(A)=50$ km

You go from A $\rightarrow$ B (cost 20 km)

From B $\rightarrow$ C, heuristic says $h(B)=25$ km

Triangle inequality check:

- $h(A) \leq c(A,B) + h(B)$
- $50 \leq 20 + 25$
- $50 \leq 45$ ✗ $\rightarrow$ fails $\rightarrow$ heuristic is inconsistent.

Dominance

- If $h_2(n) \geq h_1(n)$ for all n (both admissible)
- then h_2 dominates h_1
- h_2 is better for search
- Typical search costs (average number of nodes expanded):
 - $d=12$ IDS = 3,644,035 nodes
 $A^*(h_1) = 227$ nodes
 $A^*(h_2) = 73$ nodes
 - $d=24$ IDS = too many nodes
 $A^*(h_1) = 39,135$ nodes
 $A^*(h_2) = 1,641$ nodes

Strategies for Developing Heuristics



**Relaxed Problem
Approach**



**Subproblem/Pattern
Database.**



**Greedy Approximation
/ Domain Knowledge**

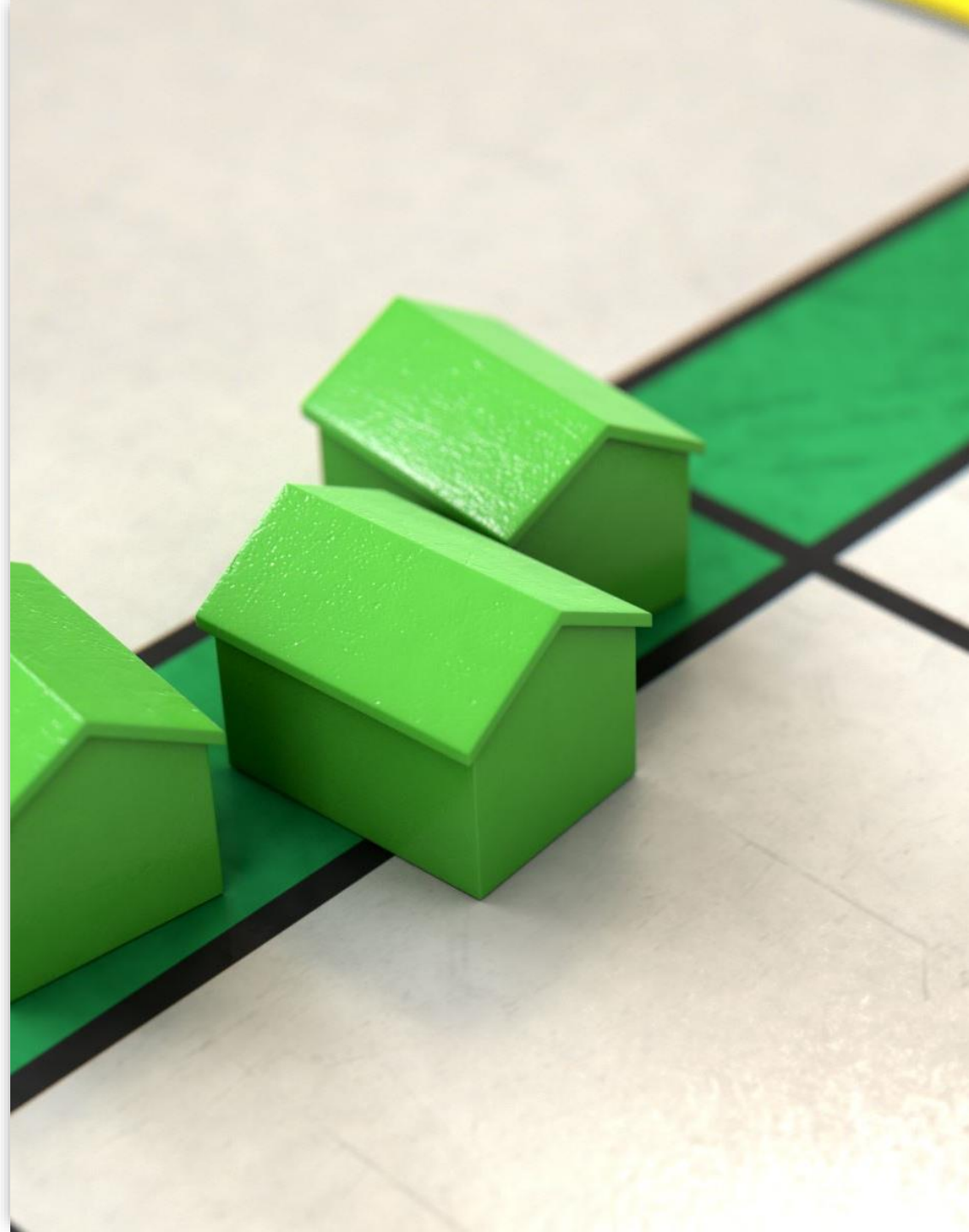
Relaxed Problem Approach

Idea:

- Remove one or more constraints from the problem to make it easier to solve.
- The solution cost to the relaxed problem is **admissible** for the original problem.

Example:

- **8-puzzle:**
Original: Tiles move only into adjacent blank space.
Relaxed: Tiles can move anywhere →
heuristic = number of misplaced tiles.



Activity:

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

- Problem: Move tiles to match the goal configuration.
- Real rules: Can only move a tile into the blank space.
 - A tile can move from square X to square Y if X is adjacent to Y and Y is blank,
- Relaxed rule:
 - (a) A tile can move from square X to square Y if X is adjacent to Y.
 - (b) A tile can move from square X to square Y if Y is blank.
 - (c) A tile can move from square X to square Y.

Activity answer (a)

compute h_2 (**Manhattan distance**): for each **numbered tile** (ignore the blank), add the horizontal + vertical distance from its current square to its goal square.

Why this matches the relaxation: with only “adjacent” required, a tile can slide one square toward its goal each move (doesn’t need the blank), so the sum of those one-square steps is exactly the cost under the relaxed rules. That makes it **admissible** for the real puzzle (it never overestimates).

Answer (b)



Under this relaxation, the **blank can swap with any tile anywhere** (not just adjacent). Compute the **minimum number of blank–tile swaps** to reach the goal from the start state. (This number is the heuristic; often called **Gaschnig's heuristic**.)



Why this matches the relaxation: because the blank can swap with any tile, each swap can place a tile directly into its correct square (or set up the next one), so the number of such swaps is exactly the optimal cost in the relaxed problem - hence **admissible**.

Relaxed Problem	Removed Constraint(s)	Derived Heuristic	Why It Works (Admissibility & Usefulness)	Where It's Weak / Not Useful
RP-1: “A tile can move to any adjacent square, ignoring whether it’s blank” (remove b, keep a)	Condition b (“destination square must be blank”)	h_2 = Manhattan distance	Measures the minimum number of single-step moves each tile must make to reach its goal, assuming no blocking by other tiles. Always $\leq$ actual cost, so admissible. Works well because Manhattan distance matches physical tile movement cost in the real puzzle.	Weak if moves can skip spaces (e.g., in a variant puzzle) — then Manhattan underestimates too much.
RP-2: “A tile can move into the blank square from anywhere” (remove a, keep b)	Condition a (“must be adjacent”)	Blank Distance heuristic.	Estimates cost based on moving tiles directly into the blank position regardless of location. Can be admissible but often weaker than Manhattan distance.	Weak because it ignores physical adjacency entirely — heuristic values tend to be small and less informative.
RP-3: “A tile can move from any square to any other square in one move” (remove a and b)	Both conditions	h_1 = Misplaced Tiles	Counts how many tiles are not in their goal position. This matches cost in the relaxed world where each misplaced tile can be put in place in one move. Admissible because real moves take ≥ 1 step per tile.	Very weak guidance — ignores how far tiles are from their correct spot. Many states have same h_1 , so poor search ordering.

Heuristic	Relaxed Problem Condition(s) Removed	How It Works	Strengths	Weaknesses
h_1 (Misplaced Tiles)	(a) & (b)	Count tiles not in goal position	Very fast; admissible	Weak estimate
h_2 (Manhattan Distance)	(b)	Sum of row+col distances	Stronger; still fast	Ignores blocking
Blank Distance	(a)	Any tile moves to blank instantly	Simple; admissible	Too coarse; ignores distances
Gaschnig's	(a)	Simulates swaps with tile that belongs in blank position	More accurate than h_2 in some cases	Costlier to compute

Subproblem/Pattern Database

Idea:

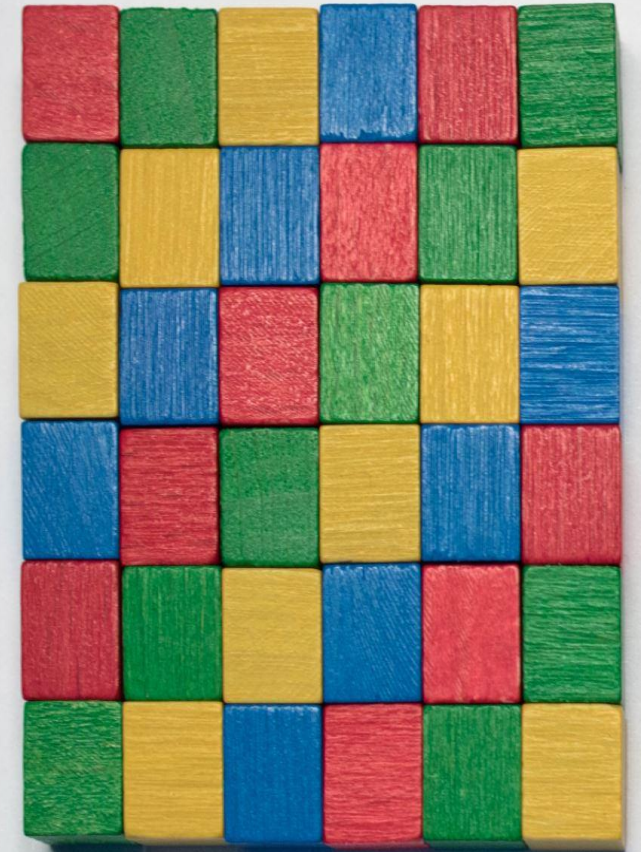
- Break the problem into smaller subproblems and precompute their optimal solutions.
- Store results in a **pattern database** for lookup.

Example:

- **Sliding-tile puzzle:** Store shortest moves for subsets of tiles and sum them for the heuristic.

Activity:

- **Problem:** You run a delivery service in a small 5×5 city grid. The final delivery point (goal) is at E. To speed up route planning, you pick 3 key points in the city: A, C, and D.



Solving Activity

- Step 1: Precompute and store the shortest-path distances from each key point to E (the goal).
- Step 2: Decide how you would store this info (e.g., in a table or dictionary).
- Step 3: Decide how you'd use this during a delivery to estimate the cost from a current location.

Key Point	Distance to E (in blocks)
A	8
C	4
D	1

Using It in a Heuristic:

If the current delivery location is **B**,

Find the nearest key point (e.g., **C**) using Manhattan distance.

Estimate total cost:

$$h(n) = \text{dist}(B, C) + \text{precomputed}[C]$$

Example:

Distance $B \rightarrow C = 2$

Precomputed distance from C to E = 4 blocks

Heuristic estimate $h(n) = 2 + 4 = 6$

A hand with a light-colored sleeve points at a specific location on a dense, multi-colored transit map. The map features various colored lines (red, blue, green, yellow, orange) representing different transit routes, with numerous station names and connecting lines visible.

Greedy Approximation / Domain Knowledge

Idea:

- Use problem-specific insights to guide the search.
- Often uses approximations that are fast to compute but based on real-world facts.

Example:

- **Map navigation:** Straight-line distance between two cities (Euclidean distance) as $h(n)$

Activity:

- **Problem:** You're building a food delivery route planner in a hilly city.
- **Task:** Propose a heuristic that factors in both straight-line distance and an estimated slope penalty.
- **Output:** Write your $h(n)$ formula and explain why it might be more effective than pure distance.

Activity Solution

- **Heuristic Formula**

$h(n) = d(n, \text{goal}) + 0.1 \times s(n, \text{goal})$ Where:

- $d(n, \text{goal})$ = straight-line (Euclidean) distance in meters
- $s(n, \text{goal})$ = elevation gain in meters
- 0.1 = slope penalty factor (1 unit cost per 10 meters elevation gain)

- **Explanation**

- Straight-line distance alone underestimates travel time because it ignores steep climbs.
- Adding a slope penalty makes the heuristic more **realistic** and better predicts travel time in hilly cities.
- This still satisfies admissibility if slope penalties are never underestimated.

Best First Search

```
function BEST-FIRST-SEARCH(problem, f) returns a solution node or failure  
  node  $\leftarrow$  NODE(STATE=problem.INITIAL)  
  frontier  $\leftarrow$  a priority queue ordered by f, with node as an element  
  reached  $\leftarrow$  a lookup table, with one entry with key problem.INITIAL and value node  
  while not IS-EMPTY(frontier) do  
    node  $\leftarrow$  POP(frontier)  
    if problem.IS-GOAL(node.STATE) then return node  
    for each child in EXPAND(problem, node) do  
      s  $\leftarrow$  child.STATE  
      if s is not in reached or child.PATH-COST < reached[s].PATH-COST then  
        reached[s]  $\leftarrow$  child  
        add child to frontier  
  return failure
```

```
function EXPAND(problem, node) yields nodes  
  s  $\leftarrow$  node.STATE  
  for each action in problem.ACTIONS(s) do  
    s'  $\leftarrow$  problem.RESULT(s, action)  
    cost  $\leftarrow$  node.PATH-COST + problem.ACTION-COST(s, action, s')  
    yield NODE(STATE=s', PARENT=node, ACTION=action, PATH-COST=cost)
```

- *node*.STATE: the state to which the node corresponds;
- *node*.PARENT: the node in the tree that generated this node;
- *node*.ACTION: the action that was applied to the parent's state to generate this node
- *node*.PATH-COST: the total cost of the path from the initial state to this node. In mathematical formulas, we use $g(\text{node})$ as a synonym for PATH-COST.

Best-first search

- Idea: use an evaluation function $f(n)$ for each node
 - estimate of "desirability"
 - Expand the most desirable unexpanded node
- Implementation:
 - *Fringe(frontier)* is a queue sorted in decreasing order of desirability
- Special cases:
 - greedy best-first search
 - A* search

```
function BEST-FIRST-SEARCH(problem, f) returns a solution node or failure
  node ← NODE(STATE=problem.INITIAL)
  frontier ← a priority queue ordered by f, with node as an element
  reached ← a lookup table, with one entry with key problem.INITIAL and value node
  while not IS-EMPTY(frontier) do
    node ← POP(frontier)
    if problem.IS-GOAL(node.STATE) then return node
    for each child in EXPAND(problem, node) do
      s ← child.STATE
      if s is not in reached or child.PATH-COST < reached[s].PATH-COST then
        reached[s] ← child
        add child to frontier
  return failure
```

```
function EXPAND(problem, node) yields nodes
  s ← node.STATE
  for each action in problem.ACTIONS(s) do
    s' ← problem.RESULT(s, action)
    cost ← node.PATH-COST + problem.ACTION-COST(s, action, s')
    yield NODE(STATE=s', PARENT=node, ACTION=action, PATH-COST=cost)
```

Breaking down a Best-First Search

- We need a data structure to store the frontier. The appropriate choice is a queue of some kind, because the operations on a frontier are:
 - IS-EMPTY(frontier) returns true only if there are no nodes in the frontier.
 - POP(frontier) removes the top node from the frontier and returns it.
 - TOP(frontier) returns (but does not remove) the top node of the frontier.
 - ADD(node, frontier) inserts node into its proper place in the queue.
- A **priority queue** first pops the node with the minimum cost according to some evaluation function, f . It is used in best-first search.
- A FIFO queue or first-in-first-out **queue** first pops the node that was added to the queue first; we shall see it is used in breadth-first search.
- A LIFO queue or last-in-first-out queue (also known as a **stack**) pops first the most Stack recently added node; we shall see it is used in depth-first search.

Greedy best-first search

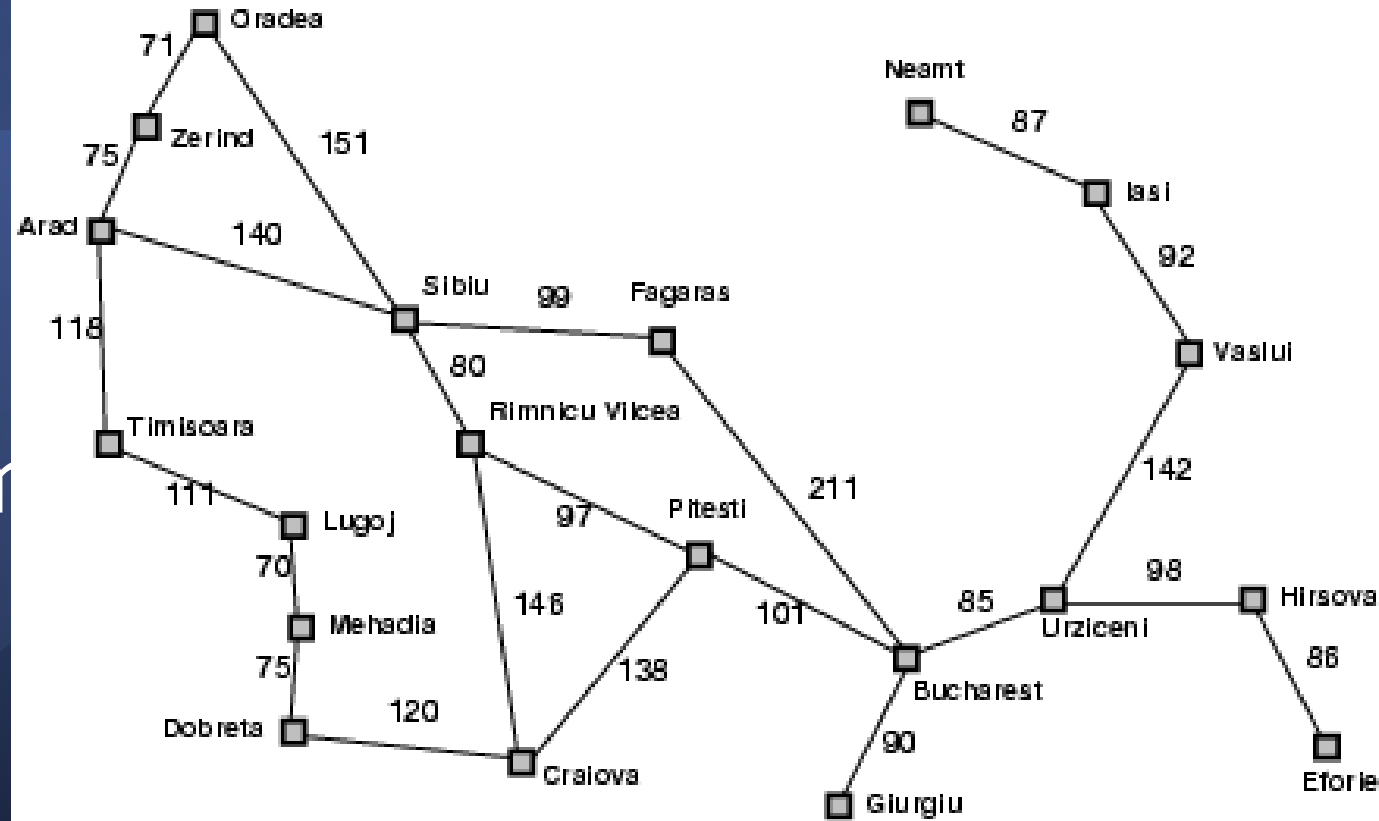
Evaluation function $h(n)$ (heuristic)

Estimate of cost from n to the closest goal

E.g., $h_{\text{SLD}}(n)$ = straight-line distance from n to Bucharest

Greedy search expands the node that appears to be closest to goal

Romania with step costs in km



Straight-line distance
to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

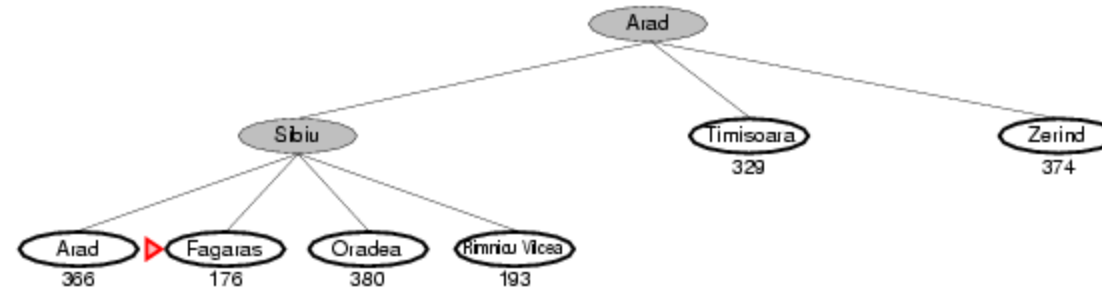
Greedy best-first search example



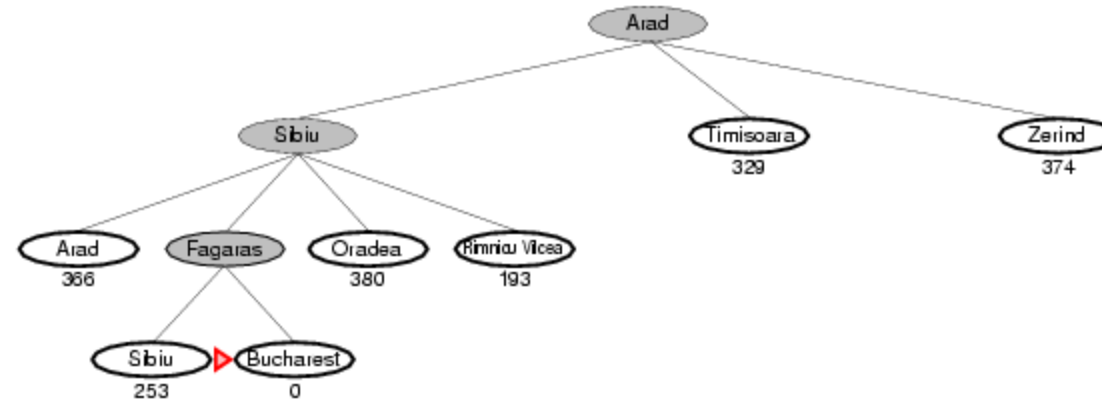
Greedy best-first search example



Greedy best-first search example



Greedy best-first search example



Properties of greedy best-first search

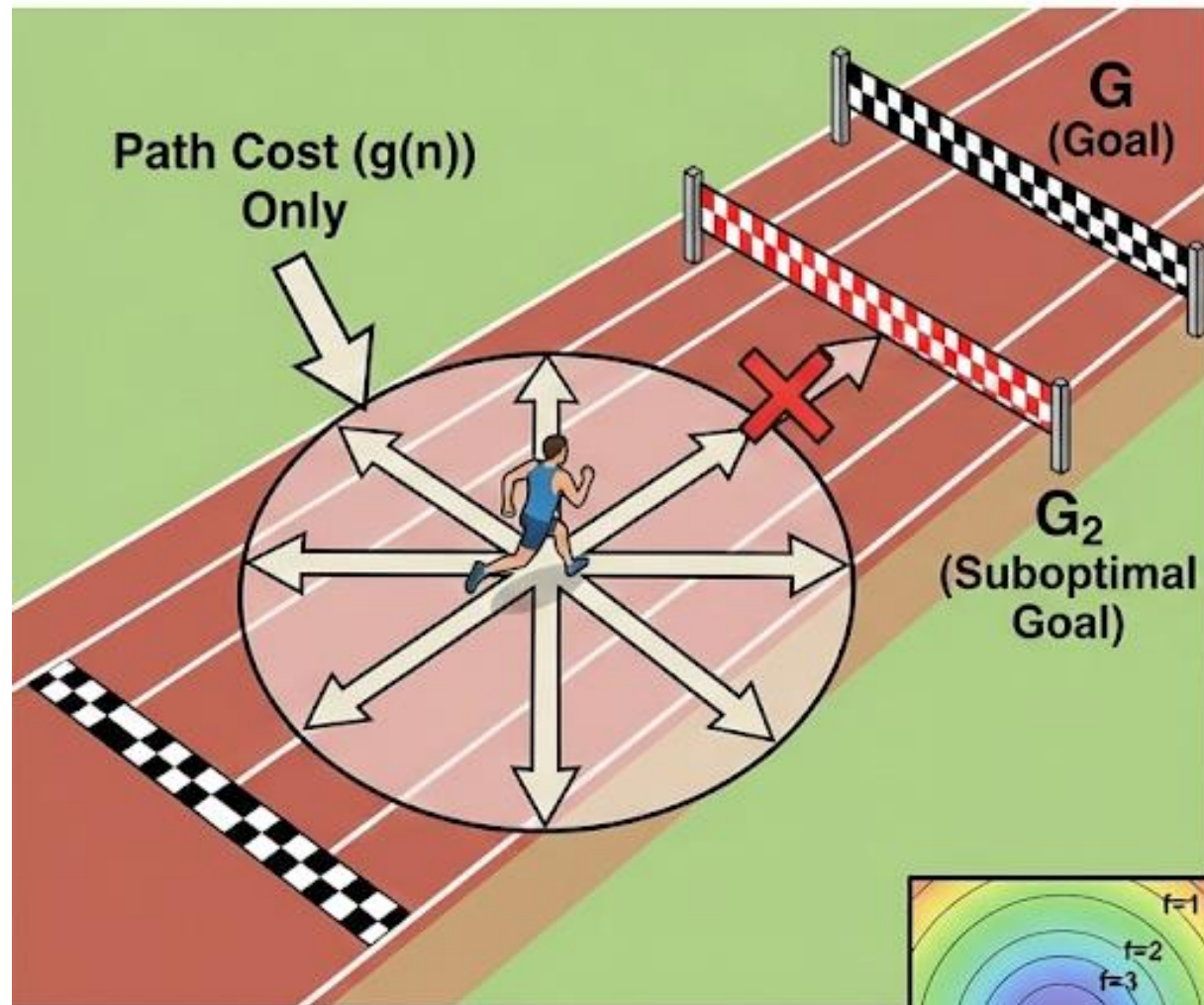
Complete? No – can get stuck in loops, e.g., lasi $\rightarrow$ Neamt $\rightarrow$ lasi $\rightarrow$ Neamt $\rightarrow$

Time? $O(b^m)$, but a good heuristic can give dramatic improvement

Space? $O(b^m)$ -- keeps all nodes in memory

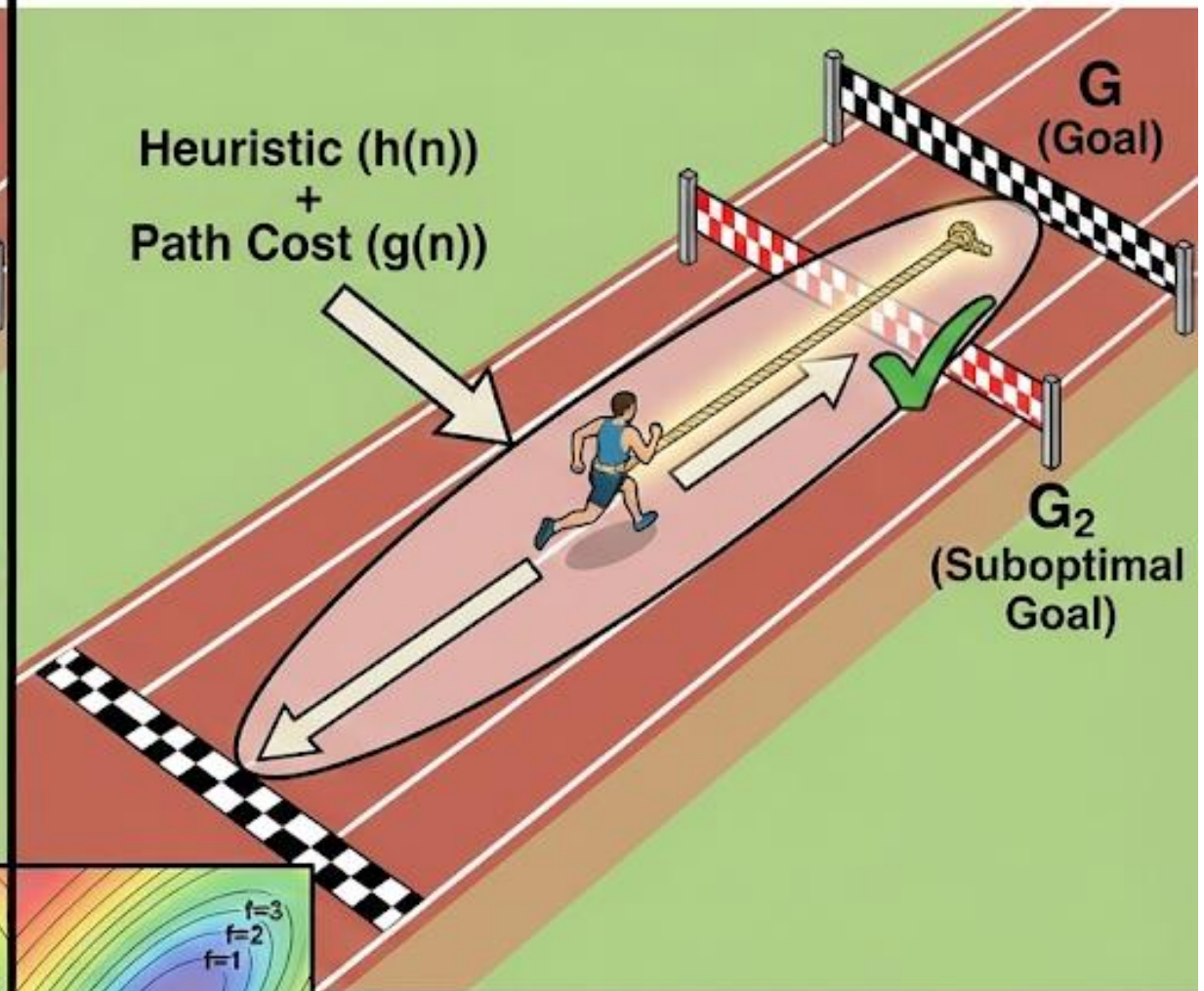
Optimal? No

Standard Search (BFS/Dijkstra) - Blind Run

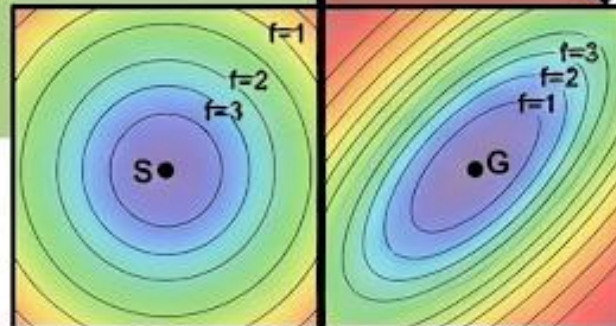


Explores equally in all directions.
Can be distracted by closer,
wrong goals (**G₂**).

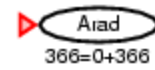
A* Search - Focused Run



Heuristic (**h**) pulls towards true goal (**G**).
Path cost (**g**) penalizes deviation to **G₂**.
Guarantees finding optimal **G**.



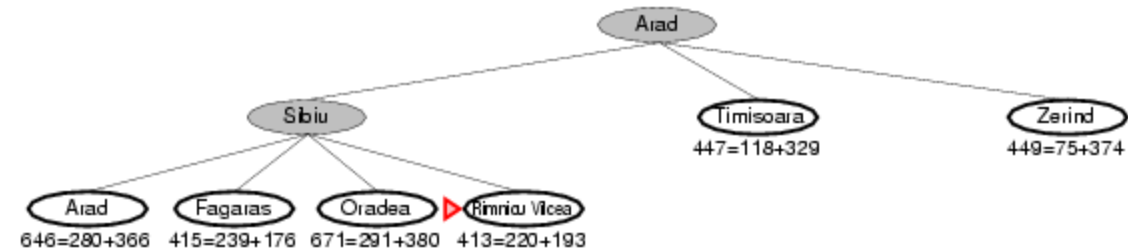
A* search example



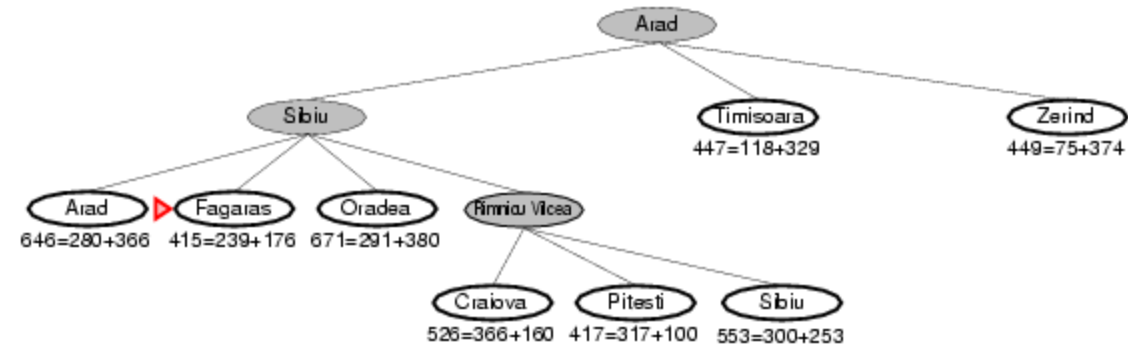
A* search example



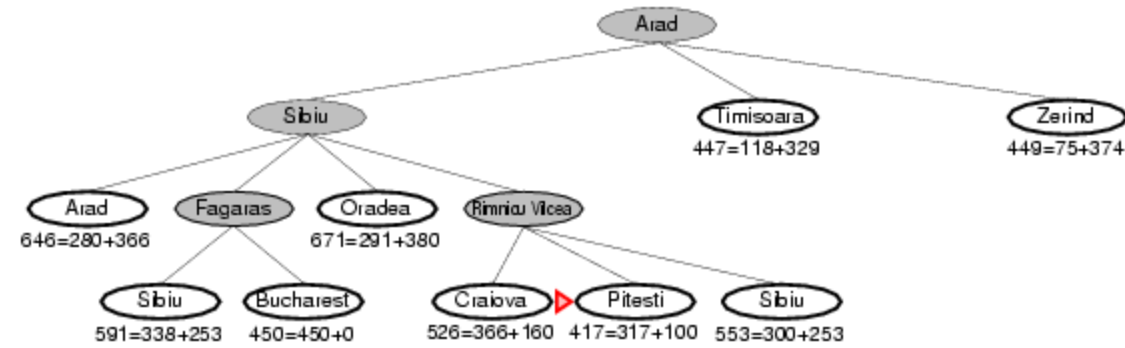
A* search example



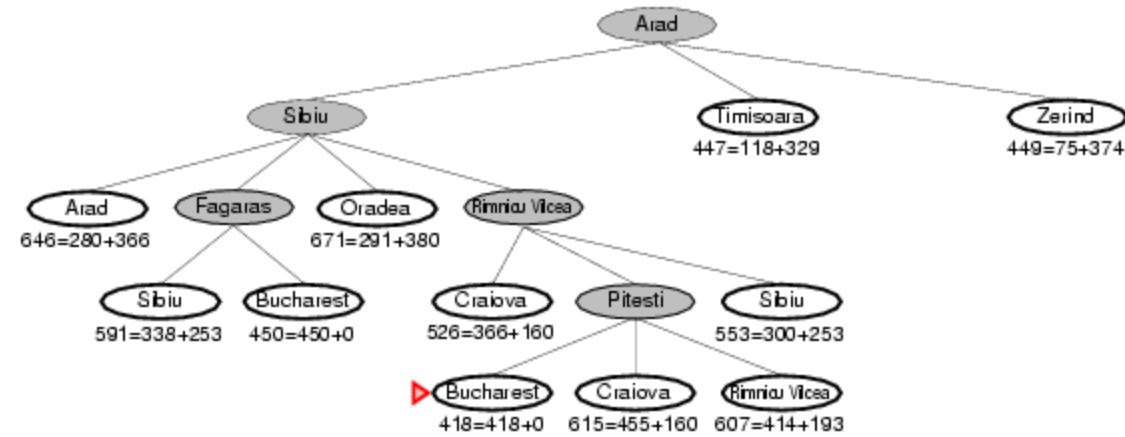
A* search example



A* search example

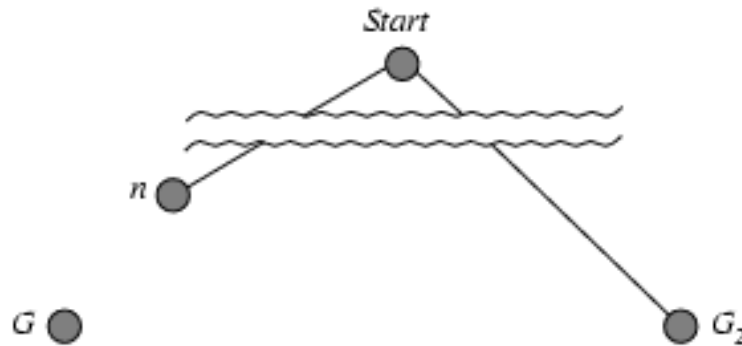


A* search example



Optimality of A^*

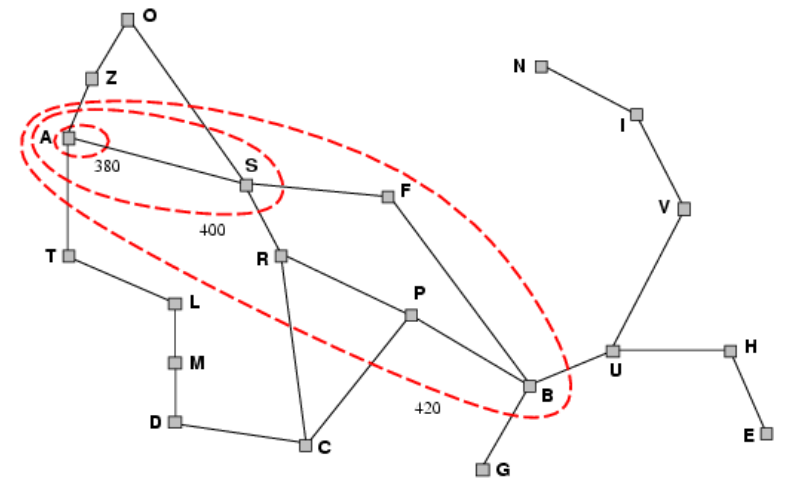
- Suppose some suboptimal goal G_2 has been generated and is in the fringe(frontier). Let n be an unexpanded node in the fringe such that n is on a shortest path to an optimal goal G .



- $f(G_2) = g(G_2)$ since $h(G_2) = 0$
- $g(G_2) > g(G)$ since G_2 is suboptimal
- $f(G) = g(G)$ since $h(G) = 0$
- $f(G_2) > f(G)$ from above
- Hence $f(G_2) > f(n)$, and A^* will never select G_2 for expansion

Optimality of A^*

- A^* expands nodes in order of increasing f value
- Gradually adds " f -contours" of nodes
- Contour i has all nodes with $f=f_i$, where $f_i < f_{i+1}$



Properties of A^*

Complete? Yes (unless there are infinitely many nodes with $f \leq f(G)$)

Time? Exponential

Space? Keeps all nodes in memory

Optimal? Yes